

React Component Lifecycle Explained

A simple guide to understanding how components are born, updated, and removed in React

💡 The Real-World Analogy: Human Life Stages

Think of a React component like a human being! It is **born** (Mounting), grows up and changes when things happen (Updating), and eventually **leaves the page** (Unmounting). React gives us special hooks to run our code at each stage.

1. The Three Main Phases

🌱 1. Mounting

"Birth of Component"

This is when a component is created and put onto the screen (DOM) for the very first time.

🔄 2. Updating

"Growing / Changing"

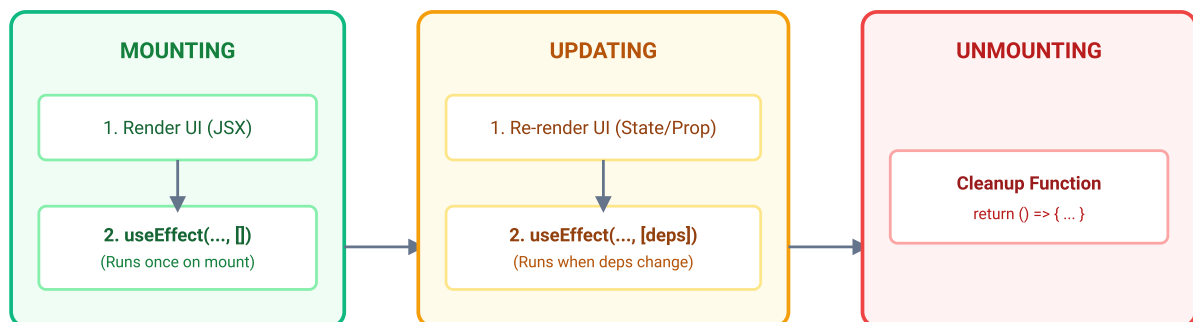
This happens whenever a component's state or props change, causing it to re-render to reflect new data.

🧹 3. Unmounting

"Leaving Screen"

This is the final step when a component is removed from the DOM and cleaned up.

2. Lifecycle Flowchart



3. Handling Lifecycle with Modern React (useEffect)

In modern React, functional components use the `useEffect` hook to manage all lifecycle behavior in one simple API.

```
import React, { useState, useEffect } from 'react'; function UserProfile({ userId }) { co
```

4. How the Dependency Array Controls Execution

Syntax	When setup runs	Equivalent Lifecycle Stage
<code>useEffect(() => {})</code>	After every single render	Mounting + Every Update
<code>useEffect(() => {}, [])</code>	Only once on mount	<code>componentDidMount</code>
<code>useEffect(() => {}, [prop])</code>	On mount + when prop changes	<code>componentDidUpdate</code>
<code>useEffect(() => { return () => {} }, [])</code>	Cleanup runs on unmount	<code>componentWillUnmount</code>

5. Key Takeaways

- **Mounting:** Use empty dependency arrays `[]` for initial setup like fetching data or setting up listeners.
- **Updating:** Include dependencies `[data]` so effects re-run whenever relevant variables change.
- **Unmounting:** Always return a cleanup function from `useEffect` to remove listeners, clear intervals, or cancel pending requests to prevent memory leaks.